

SSH BRUTE-FORCE DETECTION AND AUTOMATED RESPONSE WITH WAZUH & SHUFFLE

Authentication Monitoring | Firewall Containment
Webhook Integration | Workflow Validation

Prepared by	Muhammad Qasim
Project type	Hands-on SIEM response and SOAR integration lab
Tools	Wazuh Shuffle Ubuntu Windows
Project date	7 September 2026

I integrated Wazuh with Shuffle and configured an automatic firewall response to SSH brute-force attempts. This report covers the configuration, testing and observed results.

1. Project Overview

I tested repeated failed SSH logins from Windows against Ubuntu. The aim was to detect the activity in Wazuh, automatically block the source IP and forward the alert to Shuffle.

Lab Setup

Wazuh server	Ubuntu
Workflow platform	Shuffle
Test machine	Windows
Target IP	192.168.56.118
Source IP	192.168.56.1

Wazuh monitors the SSH activity and performs the IP block through Active Response. The integration sends the associated alert to Shuffle through a webhook.

2. Shuffle Workflow and Wazuh Integration

I connected the webhook to the first node so incoming alerts could start the workflow.

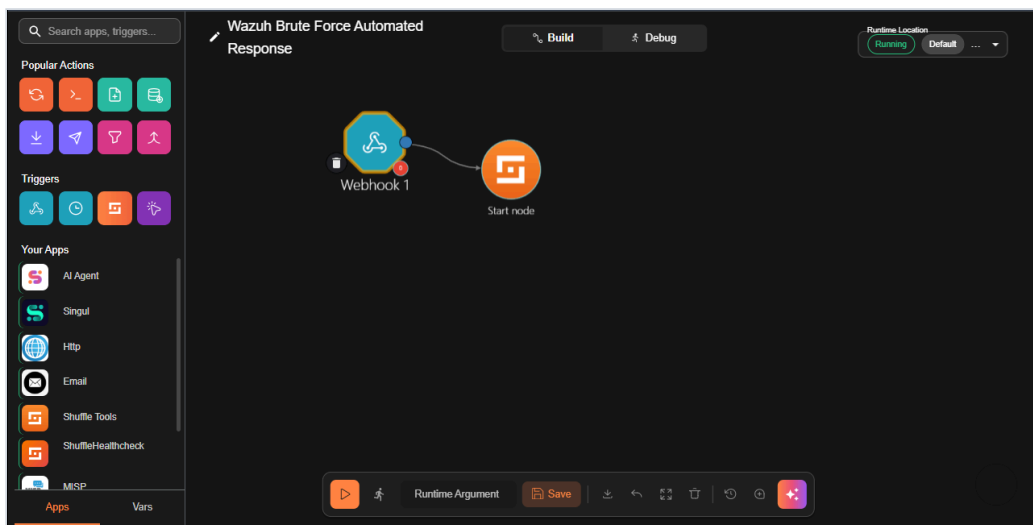


Figure 1. Webhook connected to the first node in my Shuffle workflow.

Wazuh Integration Configuration

I configured SSH log collection and added the Shuffle integration, limiting forwarding to the brute-force rule.

```

GNU nano 4.8 /var/ossec/etc/ossec.conf Modified
<ossec_config>
  <global>
    <jsonout_output>yes</jsonout_output>
    <alerts_log>yes</alerts_log>
    <email_notification>yes</email_notification>
    <smtp_server>127.0.0.1</smtp_server>
    <email_from>wazuh-alerts@wazuh.com</email_from>
    <email_to>:email_to>
    <agents_disconnection_time>15m</agents_disconnection_time>
  </global>

  <integration>
    <name>shuffle</name>
    <hook_url>|
    <rule_id>5763</rule_id>
    <alert_format>json</alert_format>
  </integration>

  <localfile>
    <log_format>syslog</log_format>
    <location>/var/log/auth.log</location>
  </localfile>

```

Figure 2. My SSH log collection and alert forwarding settings on Ubuntu.

3. Local Response Configuration

I linked the default firewall response to the detection rule and configured a three-minute timeout for the block.

```

GNU nano 4.8 /var/ossec/etc/ossec.conf
  <executable>yara.bat</executable>
  <timeout_allowed>no</timeout_allowed>
</command>

<command>
  <name>firewall-drop</name>
  <executable>firewall-drop</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>

<active-response>
  <disabled>no</disabled>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>5763</rules_id>
  <timeout>180</timeout>
</active-response>

```

Figure 3. Default firewall response configured with a short timeout for testing.

4. Webhook Connectivity Test

To check the connection, I sent a small sample event from the Ubuntu terminal. This helped me test the webhook without having to generate another SSH alert each time. Once the request was accepted, I opened Shuffle to check what had arrived.

```
root@qasim:/home/ubuntu# curl -i -X POST
> -H 'Content-Type: application/json' \
> -d '{
>   "rule":{"id":"5763","level":10,"description":"sshd brute force trying to get
>   access"},
>   "agent":{"id":"000","name":"qasim"},
>   "data":{"srcip":"192.168.56.1"},
>   "timestamp":"2026-09-07T10:00:00Z"
> }'
HTTP/2 200
date: Mon, 07 Sep 2026 10:24:46 GMT
content-type: application/json
content-length: 73
cache-control: no-cache,no-store
expires: Thu, 01 Jan 1970 00:00:00 GMT
pragma: no-cache
```

Figure 4. Sample event sent to the Shuffle webhook.

5. Sample Execution and Source Events

I could see the sample in the execution details and inspect the values I had sent. This confirmed that the webhook test worked. It was still manually supplied data; the later test would show whether Wazuh could deliver an alert on its own.

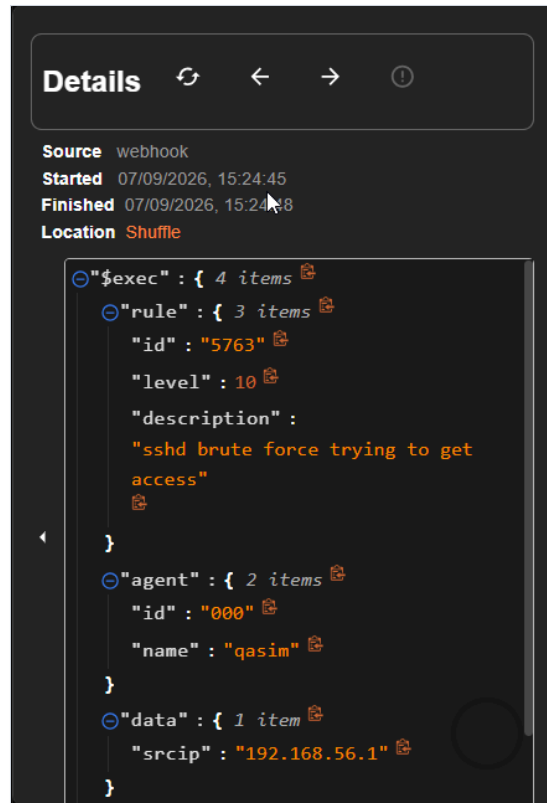


Figure 5. The sample data as received in Shuffle.

On the Ubuntu side, I checked that my failed SSH attempts were being recorded. This was useful while troubleshooting the lab: before expecting a brute-force alert, I needed to know that the underlying login failures were available for Wazuh to read.

```

Sep  7 02:56:51 qasim sshd[6382]: Failed password for ubuntu from 192.168.56.1 port 54062 ssh2
Sep  7 02:56:55 qasim sshd[6382]: Failed password for ubuntu from 192.168.56.1 port 54062 ssh2
Sep  7 02:57:01 qasim sshd[6382]: Failed password for ubuntu from 192.168.56.1 port 54062 ssh2

```

Figure 6. Authentication records from my repeated SSH login attempts.

6. Detection and Live Alert Delivery

I filtered the Wazuh dashboard for the brute-force rule and found the event from my test. The source address and Ubuntu host details let me connect it back to the login attempts. At this point, Wazuh was detecting the repeated failures as an attack.

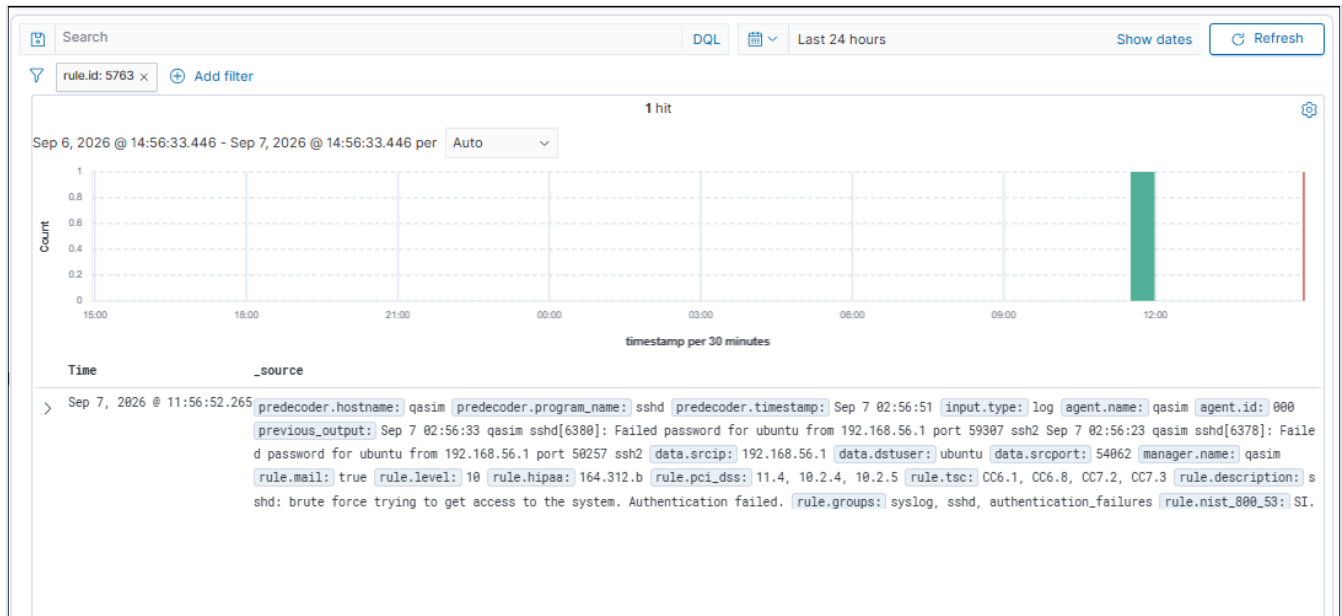


Figure 7. The brute-force detection in Wazuh Discover.

After another SSH test, Wazuh automatically sent the alert to Shuffle. I checked the execution details against the test activity to confirm that the integration worked.

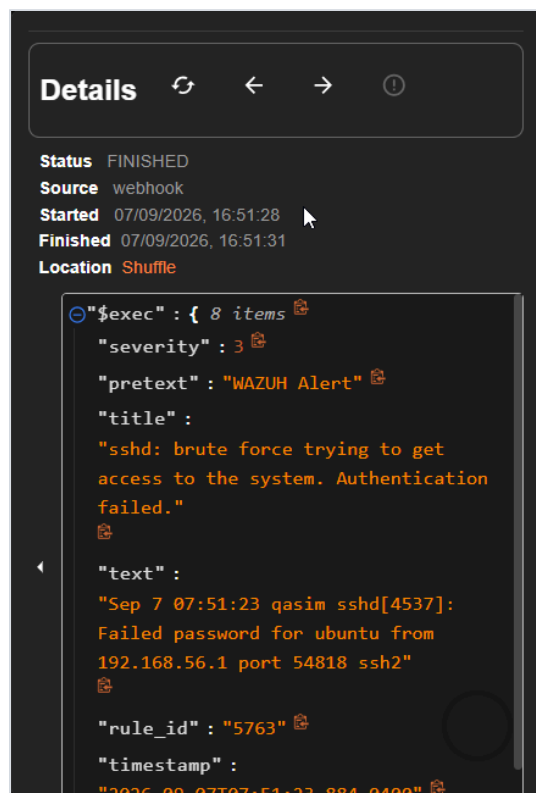


Figure 8. Live integration result after a new SSH test.

7. Response Execution

From the Ubuntu console, I checked the response log to see what happened after detection. It showed the firewall action running for the source used in my SSH test. The associated login event also matched the activity forwarded to Shuffle.

```
root@qasim:/home/ubuntu# tail -n 0 -f /var/ossec/logs/active-responses.log
2026/09/07 07:51:23 active-response/bin/firewall-drop: Starting
2026/09/07 07:51:23 active-response/bin/firewall-drop: {"version":1,"origin":{"name":"undefined","module":"wazuh-execd"},"command":"add","parameters":{"extra_args":[],"alert":{"timestamp":"2026-09-07T07:51:23.884-0400","rule":{"level":10,"description":"sshd: brute force trying to get access to the system. Authentication failed."},"id":"5763","mitre":{"id":["T1110"],"tactic":["Credential Access"],"technique":["Brute Force"]},"frequency":3,"firedtimes":2,"mail":true,"groups":["syslog","sshd","authentication_failures"],"gdpr":["IV_35.7.d","IV_32.2"],"hipaa":["164.312.b"],"nist_800_53":["SI.4","AU.14","AC.7"],"pci_dss":["11.4","10.2.4","10.2.5"],"tsc":["CC6.1","CC6.8","CC7.2","CC7.3"]},"agent":{"id":"000","name":"qasim"},"manager":{"name":"qasim"},"id":"1788781883.58583","previous_output":"Sep  7 07:51:20 qasim sshd[4535]: Failed password for ubuntu from 192.168.56.1 port 53508 ssh2\nSep  7 07:51:19 qasim sshd[4535]: Failed password for ubuntu from 192.168.56.1 port 53508 ssh2","full_log":"Sep  7 07:51:23 qasim sshd[4537]: Failed password for ubuntu from 192.168.56.1 port 54818 ssh2","predecoder":{"program_name":"sshd","timestamp":"Sep  7 07:51:23","hostname":"qasim"},"decoder":{"parent":"sshd","name":"sshd"},"data":{"srcip":"192.168.56.1","srcport":"54818","dstuser":"ubuntu"},"location":"/var/log/auth.log"},"program":"active-response/bin/firewall-drop"}}
2026/09/07 07:51:23 active-response/bin/firewall-drop: {"version":1,"origin":{"name":"firewall-drop","module":"active-response"},"command":"check_keys","parameters":{"keys":["192.168.56.1"]}}
2026/09/07 07:51:23 active-response/bin/firewall-drop: {"version":1,"origin":{"name":"undefined","module":"wazuh-execd"},"command":"continue","parameters":{"extra_args":[],"alert":{"timestamp":"2026-09-07T07:51:23.884-0400","rule":{"level":10,"description":"sshd: brute force trying to get access to the system. Authentication failed."},"id":"5763","mitre":{"id":["T1110"],"tactic":["Credential Access"],"technique":["Brute Force"]},"frequency":3,"firedtimes":2,"mail":true,"groups":["syslog","sshd","authentication_failures"],"gdpr":["IV_35.7.d","IV_32.2"],"hipaa":["164.312.b"],"nist_800_53":["SI.4","AU.14","AC.7"],"pci_dss":["11.4","10.2.4","10.2.5"],"tsc":["CC6.1","CC6.8","CC7.2","CC7.3"]},"agent":{"id":"000","name":"qasim"},"manager":{"name":"qasim"},"id":"1788781883.58583","previous_output":"Sep  7 07:51:20 qasim sshd[4535]: Failed password for ubuntu from 192.168.56.1 port 53508 ssh2\nSep  7 07:51:19 qasim sshd[4535]: Failed password for ubuntu from 192.168.56.1 port 53508 ssh2","full_log":"Sep  7 07:51:23 qasim sshd[4537]: Failed password for ubuntu from 192.168.56.1 port 54818 ssh2","predecoder":{"program_name":"sshd","timestamp":"Sep  7 07:51:23","hostname":"qasim"},"decoder":{"parent":"sshd","name":"sshd"},"data":{"srcip":"192.168.56.1","srcport":"54818","dstuser":"ubuntu"},"location":"/var/log/auth.log"},"program":"active-response/bin/firewall-drop"}}
2026/09/07 07:51:23 active-response/bin/firewall-drop: Ended
```

Figure 9. Local response log for the source used in the test.

This is where I confirmed the role of each tool: Wazuh executed the block, while Shuffle received the alert. I also checked the effect from Windows, shown on the next page, because a workflow result alone would not prove that traffic had been blocked.

8. Blocking Result

I started with a ping to check that Windows could reach the Ubuntu VM, then made SSH login attempts with incorrect passwords. During the test, the SSH connection timed out and the following pings stopped receiving replies. I used this alongside the response log to check that the block had taken effect.

```
C:\Windows\System32>ping 192.168.56.118

Pinging 192.168.56.118 with 32 bytes of data:
Reply from 192.168.56.118: bytes=32 time=3ms TTL=64
Reply from 192.168.56.118: bytes=32 time=2ms TTL=64
Reply from 192.168.56.118: bytes=32 time=2ms TTL=64
Reply from 192.168.56.118: bytes=32 time<1ms TTL=64

Ping statistics for 192.168.56.118:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 3ms, Average = 1ms

C:\Windows\System32>ssh ubuntu@192.168.56.118
+-----+
| LINUXVMIMAGES.COM |
+-----+
User Name: ubuntu

ubuntu@192.168.56.118's password:
Permission denied, please try again.
ubuntu@192.168.56.118's password:
Permission denied, please try again.
ubuntu@192.168.56.118's password:
Permission denied (publickey,password).

C:\Windows\System32>ssh ubuntu@192.168.56.118
+-----+
| LINUXVMIMAGES.COM |
+-----+
User Name: ubuntu

ubuntu@192.168.56.118's password:
Permission denied, please try again.
ubuntu@192.168.56.118's password:
Permission denied, please try again.
ubuntu@192.168.56.118's password:
Permission denied, please try again.
ssh_dispatch_run_fatal: Connection to 192.168.56.118 port 22: Connection timed out

C:\Windows\System32>ping 192.168.56.118

Pinging 192.168.56.118 with 32 bytes of data:
Request timed out.
Request timed out.
Request timed out.
```

Figure 10. Windows connection checks before and during the IP block.

The response affected traffic from the source IP, including both SSH and ping.

I captured the blocked state here. I did not take a separate screenshot of access returning after the configured timeout.

9. Results and Conclusion

By the end of the test, Wazuh was detecting the failed-login pattern and blocking my test source, and Shuffle was receiving the related alert. The screenshots above cover the setup, the sample webhook test, and the final SSH test.

What I checked	Result
SSH log collection	The failed login attempts were recorded on Ubuntu.
Brute-force detection	The related alert appeared in the Wazuh dashboard.
Sample webhook request	The sample reached Shuffle and started an execution.
Automatic alert forwarding	Wazuh delivered an alert after the live SSH test.
IP blocking	The local response ran, and Windows lost access during the test.
Block expiry	Configured for three minutes; recovery was not separately captured.

What I Learned

The main lesson was to check each part separately. A failed login in the system log, a Wazuh alert, a Shuffle execution and a firewall block all tell me something different. Checking them in that order helped me understand where the data was going and which component was responsible for the result.

I also learned to separate alert integration from response control. My workflow receives the alert, but the block is triggered by Wazuh's own configuration. I have not added notification or reputation-checking steps to this version of the workflow.

Screenshots have been cropped for readability, with webhook details, the personal notification address and banner credentials masked. The event data and results are unchanged.